

TECHNICAL PROJECT REPORT

SOC Lab / Blue Team — Detection, Analysis, and Response

Samuel Calderon Candela

Case study: Incident 001

Environment: Wazuh 4.12.0 + Sysmon on a Windows endpoint

Date of the analyzed incident: August 25, 2026 (UTC)

Document classification: Technical portfolio — SOC Analyst / Junior Blue Team

1. Introduction

This document presents the technical report of the homelab SOC / Blue Team project, whose objective was to build a functional monitoring, detection, and security analysis environment using Wazuh as the SIEM and Sysmon as the telemetry source on a Windows endpoint. The project reproduces, at a reduced scale, the real workflow of a Tier 1 SOC analyst: telemetry generation, detection, triage, analysis, correlation, timeline reconstruction, impact assessment, and documentation.

The analytical core of the report is Incident 001, a controlled and authorized simulation of a suspicious execution chain (Notepad -> CMD -> PowerShell -> encoded PowerShell -> outbound network connection). The remaining sections document the technical context needed to understand that incident.

2. Project Objective

The overall objective of the project was to build and operate a hands-on SOC lab environment using Wazuh and a Windows endpoint instrumented with Sysmon, demonstrating the SOC analyst's full cycle:

- Generate telemetry
- Detect
- Perform triage
- Analyze
- Correlate
- Reconstruct
- Evaluate
- Respond
- Document

The project aims to demonstrate skills relevant to SOC Analyst Tier 1, Junior Blue Team Analyst, Security Operations, Incident Response, and Detection Engineering roles, with a focus on Windows endpoint monitoring.

The goal was NOT to become a Wazuh administrator or to build an enterprise-grade SIEM. The goal was to demonstrate the ability to interpret real telemetry, investigate alerts, correlate process behavior, reconstruct an attack chain, identify suspicious activity, map it to MITRE ATT&CK, perform triage, and document a defensible security assessment.

The project's methodology prioritizes hands-on investigation and real telemetry over purely theoretical learning.

3. Scope

The project's scope covers the deployment of a Wazuh server, instrumentation of a Windows endpoint with Sysmon and the Wazuh Agent, configuration of local detection rules, generation of a legitimate

activity baseline, and execution of a controlled attack simulation (Incident 001) using Kali Linux as the offensive platform.

Advanced Wazuh/OpenSearch administration, building a production-grade SIEM architecture, and executing additional offensive techniques beyond those needed to validate the defined detection flow are outside the scope.

4. Lab Architecture

The lab consists of three main elements: the SIEM server (Wazuh), the monitored Windows endpoint, and the offensive simulation platform (Kali Linux).

4.1 Logical Diagram

```
Kali Linux (192.168.1.21) --TCP 4444--> WIN10-SOC-ENDPOINT01\nWIN10-SOC-ENDPOINT01 (Sysmon + Wazuh Agent) --telemetry--> Wazuh Manager / Indexer / Dashboard (Ubuntu Server)
```

5. Environment Configuration

5.1 SIEM / Server

- Ubuntu Server 24.04.4
- Wazuh Manager, Wazuh Indexer/OpenSearch, and Wazuh Dashboard 4.12.0
- Approximately 3.5 GiB of RAM, 2 CPU cores, configured swap

5.2 Windows Endpoint

- Hostname: WIN10-SOC-ENDPOINT01
- Wazuh Agent installed
- Sysmon installed and configured as the primary telemetry source
- Primary asset monitored throughout the project

5.3 Offensive Simulation Platform

Kali Linux was used exclusively as the controlled attack/simulation platform within the lab. During Incident 001, Kali operated from the IP address 192.168.1.21, with a controlled TCP listener on port 4444.

6. Telemetry Sources

The primary telemetry source used in the investigation is Sysmon Event ID 1 (process creation), which was successfully ingested end-to-end into Wazuh. The available and used fields include Image, ParentImage, CommandLine, ParentCommandLine, User, ParentUser, IntegrityLevel, ProcessId, ParentProcessId, ProcessGuid, ParentProcessGuid, SHA256, and timestamps (UTC).

ProcessGuid and ParentProcessGuid are especially relevant to this project: unlike ProcessId/ParentProcessId, which the operating system can reuse over time, GUIDs are unique

identifiers generated by Sysmon for each process, making them the only reliable basis for reconstructing parent/child process relationships throughout an investigation.

Sysmon Event ID 3 (network connection) was enabled and confirmed at the Windows level, but was not successfully ingested by Wazuh 4.12.0 in this environment. This limitation is documented in section 11, which explains how it was compensated for during the Incident 001 investigation through direct validation on the endpoint and on the controlled remote host.

7. Detection Engineering

The lab's detection engineering focused on creating local Wazuh rules over Sysmon EID 1 events, aimed at detecting parent/child process relationships considered of interest to a SOC analyst (for example, command interpreters spawned by non-administrative applications, or the use of obfuscation parameters in PowerShell). The rule design prioritizes behavior-based detection (process relationships, command-line parameters) over detection based solely on file names.

8. Configured Wazuh Rules

The following local rules were configured:

Rule	Name	Purpose
100100	SOC Lab - Notepad process creation	Detect the creation of the notepad.exe process.
100102	SOC Lab - PowerShell executed with encoded command	Detect PowerShell execution with the -EncodedCommand parameter.
100104	SOC Lab - PowerShell spawned by CMD	Detect PowerShell when spawned by cmd.exe.
100105	SOC Lab - CMD spawned by Notepad	Detect cmd.exe when spawned by notepad.exe.
100106	SOC Lab - tuning exception	Reduce a legitimate false positive generated by Wazuh Agent activity related to net.exe.

The functional validation of rules 100102, 100104, and 100105 is demonstrated solely through the Incident 001 evidence (sections 10.5 to 10.8), and is not considered confirmed merely by their existence in the configuration.

9. Legitimate Activity Baseline

Prior to the execution of Incident 001, a legitimate Notepad execution was observed, initiated by the user through Windows Explorer. The observed process relationship was explorer.exe -> notepad.exe, which triggered rule 100100.

Image	C:\Windows\System32\notepad.exe
Parent Process	C:\Windows\explorer.exe

CommandLine	"C:\WINDOWS\system32\notepad.exe"
ParentCommandLine	C:\WINDOWS\Explorer.EXE
User / Parent User	DESKTOP-H9IT5UI\WIN-CLIENT01 (both)
IntegrityLevel	Medium
Company (signature)	Microsoft Corporation
SHA256	DA5807BB0997CC6B5132950EC87EDA2B33B1AC4533CF1F7A22A6F3B576ED7C5B
UTC	2026-08-24 23:54:23.158
Rule triggered	100100 — SOC Lab - Notepad process creation

[legitimate Notepad alert (Rule 100100)]

Aug 24, 2026 @ 19:01:29.041 input.type: log agent.ip: 192.168.1.7 agent.name: WIN10-SOC-ENDPOINT01 agent.id: 001 manager.name: SRV-SIEM01
data.win.eventdata.originalFileName: NOTEPAD.EXE data.win.eventdata.image: C:\Windows\System32\notepad.exe
data.win.eventdata.product: Microsoft Windows Operating System data.win.eventdata.parentProcessGuid: {812dfa9a-60a1-6a8c-6004-00000001600}
data.win.eventdata.description: Notepad data.win.eventdata.logonGuid: {812dfa9a-6094-6a8c-14c2-8c0100000000}
data.win.eventdata.parentCommandLine: C:\WINDOWS\Explorer.EXE data.win.eventdata.processGuid: {812dfa9a-db58-6a8c-b906-0000000016}

Expanded document [View surrounding documents](#) [View single document](#)

[ANALYSIS] The event meets all the indicators of legitimate execution: standard system path, Microsoft Corporation signature, expected parent process (explorer.exe), same user for parent and child process, IntegrityLevel Medium, and no suspicious child processes in this event.

The central analytical point of this section is that a Wazuh alert does not automatically equate to a security incident. Rule 100100 correctly detected the creation of the Notepad process, but context analysis (path, signature, parent process, user, integrity) allowed the activity to be classified as Benign / Expected, thereby establishing the baseline against which the behavior observed in Incident 001 is compared.

10. Incident 001

Incident 001 constitutes the project's main case study and the report's most analytically in-depth component.

10.1 Incident Scenario

The conceptual attack chain planned for the simulation was: legitimate application (Notepad) -> CMD -> PowerShell -> encoded command -> controlled network activity toward Kali Linux. It is important to note that this sequence represents the scenario envisioned for the simulation and should not automatically be assumed to be the final confirmed chain; the actual reconstruction is based on the telemetry actually collected (sections 10.9 and 10.10).

10.2 Attack Simulation

The simulation was executed in an authorized and controlled manner within the lab, generating on the Windows endpoint the process sequence Notepad -> CMD -> PowerShell -> PowerShell with - EncodedCommand, culminating in the establishment of an outbound TCP connection to the controlled listener on Kali Linux (192.168.1.21:4444).

10.3 Detection

The entire chain was detected by local Wazuh rules 100105 (CMD spawned by Notepad), 100104 (PowerShell spawned by CMD), and 100102 (PowerShell with the -EncodedCommand parameter), over Sysmon EID 1 events correctly ingested into the Wazuh Manager.

10.4 Initial Triage

Given the appearance of the three alerts within a close time window and on the same host and user, the initial triage raised the review priority due to the combination of indicators: a non-administrative application spawning a command interpreter, a command interpreter spawning PowerShell, and the use of an obfuscation parameter (-EncodedCommand). This combination is consistent with execution and evasion techniques documented in MITRE ATT&CK, which justified proceeding with a detailed analysis rather than an automatic closure.

10.5 Confirmed Evidence — Stage 1: Notepad -> CMD

Image	C:\Windows\System32\cmd.exe
Parent Process (ParentImage)	C:\Windows\System32\notepad.exe
CommandLine	C:\Windows\System32\cmd.exe
ParentCommandLine	"C:\WINDOWS\system32\notepad.exe"
ProcessId / ParentProcessId	7852 / 3616
ProcessGuid	{812dfa9a-043b-6a8d-d807-000000001600}
ParentProcessGuid	{812dfa9a-fb42-6a8c-9907-000000001600}
UTC	2026-08-25 02:55:55.549
User / Parent User	DESKTOP-H9IT5UI\WIN-CLIENT01 (both)
IntegrityLevel	Medium
SHA256	BADF4752413CB0CBDC03FB95820CA167F0CDC63B597CCDB5EF43111180E088B0
Rule triggered	100105 — SOC Lab - CMD spawned by Notepad

[CMD, Rule 100105 and parent/child relationship]

```
✓ Aug 24, 2026 @ 21:55:55.675 input.type: log agent.ip: 192.168.1.7 agent.name: WIN10-SOC-ENDPOINT01 agent.id: 001 manager.name: SRV-SIEM01
data.win.eventdata.originalFileName: Cmd.Exe data.win.eventdata.image: C:\Windows\System32\cmd.exe
data.win.eventdata.product: Microsoft® Windows® Operating System data.win.eventdata.parentProcessGuid: {812dfa9a-fb42-6a8c-9907-000000001600}
data.win.eventdata.description: Windows Command Processor data.win.eventdata.logonGuid: {812dfa9a-6094-6a8c-14c2-8c0100000000}
data.win.eventdata.parentCommandLine: "C:\WINDOWS\system32\notepad.exe" data.win.eventdata.processGuid: {812dfa9a-043b-
```

[CONFIRMED EVIDENCE] A notepad.exe process (PID 3616) spawned a cmd.exe process (PID 7852) under the same user, with IntegrityLevel Medium, on the same endpoint. The relationship is confirmed by ParentProcessGuid={812dfa9a-fb42-6a8c-9907-000000001600} matching the ProcessGuid of the temporally closest baseline Notepad.

This parent/child relationship (Notepad spawning a command interpreter) is atypical for normal text editor usage, and therefore constitutes suspicious process behavior under any behavior-based detection criteria.

Rule 100105 correctly detected the relationship it was designed to identify. The activity itself was intentionally generated as part of the authorized lab simulation. It is essential to distinguish three concepts that are often conflated during triage:

- Correct detection: the rule identified exactly the behavior it was designed for (Notepad spawning CMD).
- Suspicious behavior: the observed process relationship is anomalous and would justify investigation in a production environment.
- Authorized context: in this lab, the activity was generated deliberately and with authorization, so it does not represent a real threat.

[EVALUATION] The final classification of this stage is Benign / Authorized Security Validation. Rule 100105 should not be described as a false positive, since a false positive implies the rule fired without the detected behavior actually occurring; here, the behavior did occur exactly as detected.

10.6 Confirmed Evidence — Stage 2: CMD -> PowerShell

Image	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Parent Process (ParentImage)	C:\Windows\System32\cmd.exe
ParentCommandLine	"C:\WINDOWS\system32\cmd.exe"
ProcessId / ParentProcessId	7036 / 9272
ProcessGuid	{812dfa9a-0445-6a8d-da07-000000001600}
ParentProcessGuid	{812dfa9a-0365-6a8d-d407-000000001600}
UTC	2026-08-25 02:56:05.754
User / Parent User	DESKTOP-H9IT5UI\WIN-CLIENT01 (both)
IntegrityLevel	Medium
SHA256	9785001B0DCF755EDDB8AF294A373C0B87B2498660F724E76C4D53F9C217C7A3
Rule triggered	100104 — SOC Lab - PowerShell spawned by CMD

[CMD -> PowerShell, Rule 100104]

```
✓ Aug 24, 2026 @ 21:56:07.905 input.type: log agent.ip: 192.168.1.7 agent.name: WIN10-SOC-ENDPOINT01 agent.id: 001 manager.name: SRV-SIEM01
data.win.eventdata.originalFileName: PowerShell.EXE data.win.eventdata.image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
data.win.eventdata.product: Microsoft Windows Operating System data.win.eventdata.parentProcessGuid: {812dfa9a-0365-6a8d-d407-000000001600}
data.win.eventdata.description: Windows PowerShell data.win.eventdata.logonGuid: {812dfa9a-6094-6a8c-14c2-8c0100000000}
data.win.eventdata.parentCommandLine: "C:\WINDOWS\system32\cmd.exe" data.win.eventdata.processGuid: {812dfa9a-0445-6a8d-da07-000000001600}
```

[CONFIRMED EVIDENCE] A powershell.exe process (PID 7036, ProcessGuid {812dfa9a-0445-6a8d-da07-000000001600}) was spawned by a cmd.exe process (ParentProcessGuid {812dfa9a-0365-6a8d-d407-000000001600}) 10.2 seconds after the Stage 1 event.

Correlation note: the ParentProcessGuid of this event ({812dfa9a-0365-6a8d-d407-000000001600}) does not match the ProcessGuid of the cmd.exe documented in Stage 1 ({812dfa9a-043b-6a8d-d807-000000001600}). This indicates that the cmd.exe process that spawned this PowerShell is a different cmd.exe instance from the one recorded in Stage 1, and it cannot be confirmed via GUID that both events correspond to the same cmd.exe process. This observation is revisited in greater depth in section 10.9.

The security relevance of this event does not lie in the presence of PowerShell alone (a legitimate binary that is extremely common on Windows), but in its parent process relationship: PowerShell spawned directly by a command interpreter (cmd.exe) which, in turn, does not come from an expected administrative flow. This chain of nested interpreters (Notepad-like -> CMD -> PowerShell) is a pattern more associated with script- or operator-driven command execution than with normal interactive PowerShell use from a terminal opened directly by the user.

10.7 Confirmed Evidence — Encoded PowerShell

Image	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Parent Process (ParentImage)	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
ParentCommandLine	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
ProcessId / ParentProcessId	10092 / 8124
ProcessGuid	{812dfa9a-05f7-6a8d-f207-000000001600}
ParentProcessGuid	{812dfa9a-dddb-6a8d-d606-000000001600}
UTC	2026-08-25 03:03:19.272
User / Parent User	DESKTOP-H9IT5UI\WIN-CLIENT01 (both)
IntegrityLevel	Medium
SHA256	9785001B0DCF755EDDB8AF294A373C0B87B2498660F724E76C4D53F9C217C7A3
Rule triggered	100102 — SOC Lab - PowerShell executed with encoded command

[PowerShell with -EncodedCommand, Rule 100102]

Aug 24, 2026 @ 22:03:21.089

input.type: log

agent.ip: 192.168.1.7

agent.name: WIN10-SOC-ENDPOINT01

agent.id: 001

manager.name: SRV-SIEM01

data.win.eventdata.originalFileName: PowerShell.EXE

data.win.eventdata.image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

data.win.eventdata.product: Microsoft® Windows® Operating System

data.win.eventdata.parentProcessGuid: {812dfa9a-dddb-6a8c-d606-000000001600}

data.win.eventdata.description: Windows PowerShell

data.win.eventdata.logonGuid: {812dfa9a-6094-6a8c-14c2-8c0100000000}

data.win.eventdata.parentCommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"

10.7.1 Observed CommandLine

"C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -EncodedCommand JABjAD0ATgBIAHcALQBAGIAagBLAGMAdAAGAFMAeQBzAHQAZQBtAC4ATgBIAHQALgBTAG8AY2tldHMuVABjcENsaWVudDskYy5Db25uZWNOKCcxOTIuMTY4LjEuMjEnLDQ0NDQpO1N0YXJ0LVNsZWVwIC1TZWNvbmlRZDlYwOyRjLkNs3NlKCK=

10.7.2 Command Decoding (Base64 / UTF-16LE)

\$c=New-Object System.Net.Sockets.TcpClient;\$c.Connect('192.168.1.21',4444);Start-Sleep -Seconds 60;\$c.Close()

Functional analysis of the decoded command:

- Creates a System.Net.Sockets.TcpClient object (native .NET class for outbound TCP connections).
- Establishes a connection to IP address 192.168.1.21 on TCP port 4444.
- Keeps the connection open for approximately 60 seconds (Start-Sleep).
- Closes the connection (Close).

[ANALYSIS] The decoded command does not contain any stream read/write logic (NetworkStream), remote command execution, or input/output redirection toward the connection. Therefore, the command technically establishes and maintains an outbound TCP connection, but does not implement an interactive shell or any observable command/data transfer within the script itself.

Consequently, the technically accurate description of this activity is a simulated C2-style communication via a controlled outbound TCP connection, not an interactive reverse shell, evidence of remote command execution, or an actual C2 compromise. The use of -EncodedCommand by itself constitutes a recognized obfuscation technique (the analyst cannot read the command's intent without decoding it first), regardless of the simplicity of the underlying payload.

10.8 Timeline Reconstruction

UTC	Event	Detection
23:54:23.158 (24/08)	Legitimate Notepad execution (baseline)	100100
02:55:55.549	notepad.exe spawns cmd.exe (PID 7852)	100105
02:56:05.754	cmd.exe spawns powershell.exe (PID 7036); +10.2 s	100104
03:03:19.272	powershell.exe spawns powershell.exe with -EncodedCommand (PID 10092); +7 min 13.5 s from the previous event	100102
~03:03:19 +	Outbound TCP connection to 192.168.1.21:4444, sustained ~60 s	Manual validation (netstat / Netcat) — see 10.10

[ANALYSIS] Between the 02:56:05.754 event (CMD->PowerShell) and the 03:03:19.272 event (encoded PowerShell), approximately 7 minutes and 13 seconds elapse, a significantly larger interval than the one observed between the previous events (10.2 seconds). This time difference is consistent with — and reinforces — the correlation finding in section 10.9: there is no indication that both events are part of a single, continuous, uninterrupted execution.

10.9 Critical Process Correlation Finding

This is one of the most important analytical points of Incident 001.

The conceptual scenario envisioned for the simulation (section 10.1) suggests a single continuous chain: Notepad -> CMD -> PowerShell -> encoded PowerShell, as if each process were a direct child of the

previous one in a single execution sequence. The telemetry evidence, however, does not allow that continuous chain to be confirmed when examined at the ProcessGuid/ParentProcessGuid level.

10.9.1 What the Telemetry Confirms

- notepad.exe (ProcessGuid {812dfa9a-fb42-6a8c-9907-000000001600}) spawned cmd.exe (ProcessGuid {812dfa9a-043b-6a8d-d807-000000001600}). Relationship confirmed by GUID.
- A cmd.exe process (ProcessGuid {812dfa9a-0365-6a8d-d407-000000001600}) spawned the first powershell.exe (ProcessGuid {812dfa9a-0445-6a8d-da07-000000001600}). Relationship confirmed by GUID, but this cmd.exe ProcessGuid does not match the ProcessGuid of the cmd.exe documented in Stage 1.
- A powershell.exe process (ProcessGuid {812dfa9a-dddb-6a8d-d606-000000001600}) spawned the powershell.exe with -EncodedCommand (ProcessGuid {812dfa9a-05f7-6a8d-f207-000000001600}). Relationship confirmed by GUID, but this parent PowerShell ProcessGuid does not match the ProcessGuid of the first powershell.exe documented in Stage 2 ({812dfa9a-0445-6a8d-da07-000000001600}).

10.9.2 What Was Originally Expected

The conceptual design of the simulation (section 10.1) assumed that the same cmd.exe process from Stage 1 would spawn the Stage 2 PowerShell, and that that same PowerShell would ultimately execute the encoded command, forming a single continuous branch in the process tree.

10.9.3 What the Telemetry Actually Demonstrates

The telemetry demonstrates three valid, independent parent/child relationships (Notepad->CMD confirmed by GUID; CMD->PowerShell confirmed by GUID; PowerShell->encoded PowerShell confirmed by GUID), but it does not demonstrate that these three relationships form a single, uninterrupted chain of direct descent between the first and last events. The ProcessGuids of the intermediate processes (cmd.exe and the first powershell.exe) referenced as "parent" at each hop do not match the ProcessGuids of the "child" processes documented in the previous hop.

10.9.4 What Cannot Be Confirmed

[INFERENCE] With the telemetry available, it cannot be confirmed that the powershell.exe process that executed the encoded command is a direct descendant, within a single continuous branch of the process tree, of the cmd.exe spawned by Notepad in Stage 1. It is possible that additional intermediate processes existed that were not captured in the evidence provided, or that multiple independent instances of cmd.exe/powershell.exe were opened during the simulation window. Neither of these possibilities can be confirmed or ruled out with the available data.

10.9.5 Why This Finding Matters

This finding should not be hidden or forced to fit the evidence to the scenario's original hypothesis. On the contrary, it is a practical, central demonstration of the project: it illustrates why ProcessId and ParentProcessId alone are insufficient for reliable reconstruction (since they can be reused by the operating system), and why an analyst must validate every link in a process chain via ProcessGuid/ParentProcessGuid before asserting continuity. Reporting a continuous chain without that

validation would have amounted to presenting an inference as confirmed evidence, which is explicitly outside this report's standards of rigor.

[CONCLUSION] Each individual hop in the chain (Notepad->CMD, CMD->PowerShell, PowerShell->encoded PowerShell) is confirmed by its own ProcessGuid/ParentProcessGuid pair. What is not confirmed is the continuity of a single, uninterrupted branch between the first and last events in the sequence.

10.10 Network Validation

During the execution of the encoded PowerShell, the endpoint established a TCP connection to 192.168.1.21:4444, where Kali Linux maintained an active listener via Netcat.

Since Sysmon Event ID 3 was not successfully ingested by Wazuh in this environment (sections 6 and 11), the network connection could not be validated through a Wazuh alert. To compensate for this telemetry limitation, validation was performed independently on both ends of the communication:

- On the Windows endpoint, via the `netstat -ano` command, observing the active connection to 192.168.1.21:4444 along with its associated PID.
- On Kali Linux, via direct observation of the inbound connection on the Netcat listener.

[netstat -ano output on Windows showing the active connection to 192.168.1.21:4444 and its PID]

```
C:\Users\WIN-CLIENT01>netstat -ano | findstr :4444
TCP      192.168.1.7:51926      192.168.1.21:4444      ESTABLISHED      7036
```

[CONFIRMED EVIDENCE] Wazuh did not detect this connection via Sysmon EID 3 (not ingested). Validation of the network activity was performed through manual, direct, and correlated verification on the endpoint (netstat) and on the controlled remote host (Netcat), rather than through an alert generated by the SIEM.

10.11 Correlation

The incident correlation combines four sources of evidence: the three Wazuh alerts (100105, 100104, 100102) generated over Sysmon EID 1 events on the same host and user (DESKTOP-H9IT5UI\WIN-CLIENT01) within a window of approximately 7.5 minutes, and the independent network validation (netstat on Windows + Netcat on Kali) coinciding in time with the last encoded PowerShell event.

This multi-source correlation makes it possible to reconstruct a coherent picture of the behavior observed on the endpoint, even considering that the strict continuity of the process tree branch could not be fully confirmed (section 10.9) and that Wazuh's native network telemetry was not available (section 6).

10.12 MITRE ATT&CK Mapping

Technique	Name	Justification (observed evidence)
T1059.003	Windows Command Shell	Execution of cmd.exe spawned by notepad.exe (Stage 1, Rule 100105).

T1059.001	PowerShell	Execution of powershell.exe spawned by cmd.exe and, subsequently, by another powershell.exe process (Stages 2 and 3, Rules 100104 and 100102).
T1027	Obfuscated/Compressed Files or Information	Use of the -EncodedCommand parameter with a Base64/UTF-16LE payload, which hides the command content until decoded (Rule 100102).

[EVALUATION] The outbound TCP connection to 192.168.1.21:4444 is not assigned to a specific Command and Control technique (for example, T1071 - Application Layer Protocol, or T1571 - Non-Standard Port). The decoded command only opens, maintains, and closes a TCP socket, with no identifiable application protocol, no observed command traffic, and no persistence. Assigning a specific C2 technique based on a single simple TCP connection would constitute an over-interpretation not supported by the available evidence.

10.13 Impact Assessment

The impact confirmed by the evidence is limited to the following:

- Controlled process execution within the lab endpoint (Notepad, CMD, PowerShell).
- Execution of an encoded PowerShell command (T1027 obfuscation).
- Establishment of a controlled outbound TCP connection to an authorized lab host.
- Successful generation and detection of telemetry through the local Wazuh rules.

[CONCLUSION] No evidence of persistence, privilege escalation, lateral movement, data exfiltration, ransomware activity, actual C2, or system compromise is observed. None of these impacts was demonstrated by the available telemetry and, therefore, are not asserted in this report.

10.14 Incident Classification

10.14.1 Production Perspective

If this same combination of behaviors (Notepad spawning CMD, CMD spawning PowerShell, PowerShell executing an encoded command, an outbound connection to an external host over a non-standard port, all on the same user and host) appeared unexpectedly on a production endpoint, it would justify a high-priority investigation, given the combination of execution (T1059) and obfuscation (T1027) indicators.

10.14.2 Lab Perspective

In this project, the activity was generated deliberately and with authorization as part of the simulation. Therefore, the final incident classification is Benign / Authorized Security Validation, with disposition Closed — Controlled Incident Simulation.

[EVALUATION] This classification does not equate to labeling the detections as false positives. Rules 100105, 100104, and 100102 correctly detected the behaviors they were designed for; what determines the final benign classification is the lab's authorized context, not a detection failure.

10.15 Containment / Response Recommendations

As a triage exercise, and assuming this behavior were observed in production without prior authorization context, the recommended response actions would be:

- Isolate the endpoint from the network while the legitimacy of the activity is confirmed.
- Collect and preserve the associated Sysmon EID 1 events (full ProcessGuids) for forensic analysis.
- Decode and analyze any obfuscated PowerShell command before dismissing or escalating the alert.
- Validate the outbound network connection against reputation lists and threat intelligence, and identify the process owning the socket (PID) using operating system tools if the SIEM's network telemetry is not available.
- Interview the endpoint's user/owner to confirm whether the activity was intentionally initiated.
- Escalate to a Tier 2 analyst if a legitimate origin cannot be confirmed within a reasonable timeframe.

11. Infrastructure Limitations

During the lab's implementation phase, technical difficulties typical of deploying a Wazuh/OpenSearch environment on limited resources arose, which required tuning and troubleshooting work before reaching a stable operational environment.

Likewise, although Sysmon Event ID 3 (network connection) was enabled and confirmed at the Windows operating system level, that event was not successfully ingested by Wazuh 4.12.0 in this environment. This limitation did not prevent the completion of the Incident 001 investigation: Sysmon Event ID 1 and the rest of the process-creation telemetry continued to function correctly throughout the project, and the analyst compensated for the absence of the SIEM's network telemetry through direct validation on the endpoint (netstat) and on the controlled remote host (Netcat), as documented in section 10.10.

These limitations are documented for the sake of technical transparency and do not constitute the central focus of this report.

12. Lessons Learned

- ProcessGuid and ParentProcessGuid are essential for a reliable reconstruction of process trees; ProcessId/ParentProcessId are not sufficient on their own, since the operating system reuses them over time.
- A correctly triggered alert does not automatically imply a security incident; context (baseline, authorization, expected environment behavior) is essential for the final classification.
- The absence of a specific telemetry source (in this case, Sysmon EID 3 in Wazuh) can be partially compensated for through manual, correlated validation across the systems involved, although this increases the investigative effort.
- It is preferable to explicitly document what the evidence does not allow to be confirmed, rather than forcing a continuous narrative that is not supported by the available data.

- The execution of obfuscated commands (-EncodedCommand) requires a mandatory additional decoding step before any severity assessment.

13. Conclusion

The project demonstrated the ability to operate the complete analytical cycle of a Tier 1 SOC over a real lab environment: telemetry generation via Sysmon, detection via local Wazuh rules, initial triage, detailed analysis of processes and command lines, ProcessGuid-based correlation, timeline reconstruction, MITRE ATT&CK mapping, impact assessment, and documentation of a defensible conclusion.

The project's central value does not lie in the number of alerts generated or the complexity of the simulated offensive techniques, but in the analytical discipline applied during the Incident 001 investigation: the explicit distinction between confirmed evidence, analysis, inference, and evaluation; the honest acknowledgment of a discontinuity in the ProcessGuid chain instead of forcing the original narrative; and the methodical compensation of a real telemetry limitation through cross-validation. These capabilities directly reflect the standard of rigor expected of a Tier 1 SOC Analyst / Junior Blue Team analyst in a professional environment.